Data analysis reproducible code for "A spectral confounder adjustment for spatial regression with multiple exposures and outcomes"

Anonymous

2025-12-18

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.2.3

## Warning: package 'ggplot2' was built under R version 4.2.3

## Warning: package 'tibble' was built under R version 4.2.3

## Warning: package 'tidyr' was built under R version 4.2.3

## Warning: package 'readr' was built under R version 4.2.3

## Warning: package 'purrr' was built under R version 4.2.3

## Warning: package 'dplyr' was built under R version 4.2.3

## Warning: package 'stringr' was built under R version 4.2.3

## Warning: package 'forcats' was built under R version 4.2.3

## Warning: package 'lubridate' was built under R version 4.2.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.4
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.4.4     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.0
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
source("functions_analysis.R")
```

```
## Loading required package: coda
## Loading required package: MASS
```

```
## Warning: package 'MASS' was built under R version 4.2.3
```

```
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
##     select
##
## ##
## ## Markov Chain Monte Carlo Package (MCMCpack)
## ## Copyright (C) 2003-2025 Andrew D. Martin, Kevin M. Quinn, and Jong Hee Park
## ##
## ## Support provided by the U.S. National Science Foundation
## ## (Grants SES-0350646 and SES-0350613)
## ##
```

```
## Warning: package 'splines2' was built under R version 4.2.3
```

```
## Warning: package 'plyr' was built under R version 4.2.3
```

```
## ------------------------------------------------------------------------------
## You have loaded plyr after dplyr - this is likely to cause problems.
## If you need functions from both plyr and dplyr, please load plyr first, then dplyr:
## library(plyr); library(dplyr)
## ------------------------------------------------------------------------------
##
## Attaching package: 'plyr'
##
## The following objects are masked from 'package:dplyr':
##
##     arrange, count, desc, failwith, id, mutate, rename, summarise,
##     summarize
##
## The following object is masked from 'package:purrr':
##
##     compact
```

```
## Warning: package 'LaplacesDemon' was built under R version 4.2.3
```

```
##
## Attaching package: 'LaplacesDemon'
##
## The following objects are masked from 'package:MCMCpack':
##
##     BayesFactor, ddirichlet, dinvgamma, rdirichlet, rinvgamma
```

```
##
## The following objects are masked from 'package:lubridate':
##
##     dst, interval
##
## The following object is masked from 'package:purrr':
##
##     partial

## Warning: package 'doParallel' was built under R version 4.2.3

## Loading required package: foreach

## Warning: package 'foreach' was built under R version 4.2.3

##
## Attaching package: 'foreach'
##
## The following objects are masked from 'package:purrr':
##
##     accumulate, when
##
## Loading required package: iterators
## Loading required package: parallel

## Warning: package 'fields' was built under R version 4.2.3

## Loading required package: spam

## Warning: package 'spam' was built under R version 4.2.3

## Spam version 2.10-0 (2023-10-23) is loaded.
## Type 'help( Spam)' or 'demo( spam)' for a short introduction
## and overview of this package.
## Help for individual functions is also obtained by adding the
## suffix '.spam' to the function name, e.g. 'help( chol.spam)'.
##
## Attaching package: 'spam'
##
## The following object is masked from 'package:LaplacesDemon':
##
##     rmvt
##
## The following objects are masked from 'package:base':
##
##     backsolve, forwardsolve
##
## Loading required package: viridisLite

## Warning: package 'viridisLite' was built under R version 4.2.3

##
## Try help(fields) to get started.
```

Get data ready

Read in data

```
data_nc <- read_csv("data_nc.csv")
```

```
## Rows: 797 Columns: 8
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): geometry
## dbl (7): ZCTA5CE20, RPL_theme1, RPL_theme2, RPL_theme3, RPL_theme4, TOTAL, U...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
adj.mat_nc <- readRDS("adj.mat_nc.rds")
```

Find the eigendecomposition from the adjacency matrix

```
M_nc <- diag(rowSums(adj.mat_nc))
Qmat_nc   <- M_nc - adj.mat_nc
eig_nc <- eigen(Qmat_nc)
Gamma_nc <- eig_nc$vectors
W_nc <- eig_nc$values
```

Simulate outcomes from the exposures

```
# "true" beta
sim_beta <- matrix(c(.2, .3, .4, .5, .3, .4, .5, .6, .4, .5, .2, .3, .3, .5, .2, .4, .5, .6, .1, .5), ni
# simulate outcomes from the exposures
# set all coefficients as 0.5
exposure_matrix <- data_nc %>% dplyr::select(c("RPL_theme1", "RPL_theme2", "RPL_theme3", "RPL_theme4"))

set.seed(919)
X  <- as.matrix(exposure_matrix)
Xb <- X%*%sim_beta
E  <- rnorm(nrow(X)*5, 0, 1)
sim_outcome_matrix  <- 5 + X%*%sim_beta + E

data_nc$HYPERT_avg_IncRateYear <- sim_outcome_matrix[,1]
data_nc$CHRNKIDN_avg_IncRateYear <- sim_outcome_matrix[,2]
data_nc$HYPERL_avg_IncRateYear <- sim_outcome_matrix[,3]
data_nc$CHF_avg_IncRateYear <- sim_outcome_matrix[,4]
data_nc$DIABETES_avg_IncRateYear <- sim_outcome_matrix[,5]
```

Get R objects ready for the model

```
# create Y, X, Z for analysis

exposures <- data_nc %>%
  dplyr::select(c("RPL_theme1", "RPL_theme2", "RPL_theme3", "RPL_theme4")) %>%
  dplyr::rename(theme1 = RPL_theme1,
                theme2 = RPL_theme2,
                theme3 = RPL_theme3,
                theme4 = RPL_theme4)

outcomes <- data_nc %>% dplyr::select(contains("IncRateYear")) %>% log(.)

covariates <- cbind(log(data_nc$TOTAL), data_nc$Urbanicity)

Y <- as.matrix(outcomes)
X <- as.matrix(exposures)
Z <- as.matrix(covariates)

Y_standardized <- scale(Y)
Z_standardized <- scale(Z)

Y_star <- t(Gamma_nc) %*% Y_standardized
X_star <- t(Gamma_nc) %*% X
Z_star <- t(Gamma_nc) %*% cbind(1, Z_standardized)
```

Run the MSM, naive, and OLD models

```
exposure_names <- c("Theme 1", "Theme 2", "Theme 3", "Theme 4")
outcome_names <- c("Hypertension", "CKD", "Hyperlipidemia", "CHF", "Diabetes")

set.seed(919)

B10_2 <-  bSpline(rank(W_nc)/length(W_nc),df=10, intercept = T)

iters <- 10000
burn <- 2000
thin <- 10
n_sample= (iters-burn)/thin

# MSM (full model)

# fit_svi_K10_L10_stB <- MCMC_model4_8_3_XZ(Y_star, X_star, Z_star, Qmat_nc, E=4, n=nrow(Y_star), R=5,
# saveRDS(fit_svi_K10_L10_stB, file = "fit_svi_K10_L10_stB.rds")
fit_svi_K10_L10_stB <- readRDS("fit_svi_K10_L10_stB.rds")

### naive model

# fit_svi_naive <- MCMC_model3_2_XZ(Y_star, X_star, Z_star, Qmat_nc, p=4, n=nrow(Y_star), R=5, Q=5, ite
# saveRDS(fit_svi_naive, file = "fit_svi_naive.rds")
```

```r
fit_svi_naive <- readRDS("fit_svi_naive.rds")

### OLS
R <- 5
fit_svi <- list()
for (r in 1:R){
  fit_svi[[r]] <- lm(Y_star[,r] ~ X_star + Z_star - 1)
  print(summary(fit_svi[[r]]))
}
```

```
##
## Call:
## lm(formula = Y_star[, r] ~ X_star + Z_star - 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.12392 -0.65149  0.00188  0.61124  3.05212
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## X_startheme1 -0.08992    0.18922  -0.475  0.63476
## X_startheme2  0.32569    0.15481   2.104  0.03571 *
## X_startheme3  0.56917    0.19760   2.880  0.00408 **
## X_startheme4  0.24472    0.20521   1.193  0.23340
## Z_star1      -0.62610    0.13929  -4.495    8e-06 ***
## Z_star2       0.03342    0.05536   0.604  0.54630
## Z_star3       0.01039    0.04222   0.246  0.80561
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9765 on 790 degrees of freedom
## Multiple R-squared:  0.05357,    Adjusted R-squared:  0.04519
## F-statistic: 6.389 on 7 and 790 DF,  p-value: 2.513e-07
##
##
## Call:
## lm(formula = Y_star[, r] ~ X_star + Z_star - 1)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.4273 -0.6592 -0.0137  0.6427  3.1564
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## X_startheme1  0.40619    0.18262   2.224 0.026416 *
## X_startheme2  0.34174    0.14942   2.287 0.022450 *
## X_startheme3  0.22476    0.19071   1.179 0.238936
## X_startheme4  0.74478    0.19806   3.760 0.000182 ***
## Z_star1      -1.02977    0.13444  -7.660 5.45e-14 ***
## Z_star2      -0.01584    0.05344  -0.296 0.766997
## Z_star3       0.04799    0.04075   1.178 0.239239
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 0.9425 on 790 degrees of freedom
## Multiple R-squared:  0.1184, Adjusted R-squared:  0.1106
## F-statistic: 15.16 on 7 and 790 DF,  p-value: < 2.2e-16
##
##
## Call:
## lm(formula = Y_star[, r] ~ X_star + Z_star - 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.93741 -0.67142 -0.03528  0.59985  2.68286
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## X_startheme1  0.294881   0.187096   1.576   0.1154
## X_startheme2  0.369720   0.153079   2.415   0.0160 *
## X_startheme3  0.372604   0.195383   1.907   0.0569 .
## X_startheme4  0.419062   0.202910   2.065   0.0392 *
## Z_star1      -0.872916   0.137731  -6.338 3.91e-10 ***
## Z_star2      -0.018830   0.054745  -0.344   0.7310
## Z_star3       0.003221   0.041744   0.077   0.9385
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9656 on 790 degrees of freedom
## Multiple R-squared:  0.07466,    Adjusted R-squared:  0.06646
## F-statistic: 9.106 on 7 and 790 DF,  p-value: 7.597e-11
##
##
## Call:
## lm(formula = Y_star[, r] ~ X_star + Z_star - 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.83378 -0.63122 -0.00876  0.65195  3.14154
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## X_startheme1  0.34357    0.18748   1.833   0.0673 .
## X_startheme2  0.34363    0.15340   2.240   0.0254 *
## X_startheme3  0.20610    0.19579   1.053   0.2928
## X_startheme4  0.49889    0.20333   2.454   0.0144 *
## Z_star1      -0.83313    0.13802  -6.036 2.42e-09 ***
## Z_star2      -0.02873    0.05486  -0.524   0.6007
## Z_star3       0.01210    0.04183   0.289   0.7724
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9676 on 790 degrees of freedom
## Multiple R-squared:  0.07082,    Adjusted R-squared:  0.06259
## F-statistic: 8.602 on 7 and 790 DF,  p-value: 3.433e-10
##
##
```

```
## Call:
## lm(formula = Y_star[, r] ~ X_star + Z_star - 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.97630 -0.60042  0.00866  0.65602  2.94876
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## X_startheme1  0.21625    0.18550   1.166   0.2440
## X_startheme2  0.68017    0.15177   4.482 8.51e-06 ***
## X_startheme3 -0.04281    0.19371  -0.221   0.8252
## X_startheme4  0.45845    0.20118   2.279   0.0229 *
## Z_star1      -0.76421    0.13655  -5.596 3.02e-08 ***
## Z_star2       0.01165    0.05428   0.215   0.8302
## Z_star3      -0.04122    0.04139  -0.996   0.3196
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9573 on 790 degrees of freedom
## Multiple R-squared:  0.0904, Adjusted R-squared:  0.08234
## F-statistic: 11.22 on 7 and 790 DF,  p-value: 1.377e-13
```

Compile results

MSM

The results from MSM are saved as tensor margins. Here we find the coefficients.

```
### Calculate coefficient from tensor margins

E <- 4
R <- 5
Model <- fit_svi_K10_L10_stB
n <- nrow(Y_star)
Beta <- array(0, c(n,E,R,n_sample))
for (i in 1:n_sample){
  Beta[,,,i] <- bbygamma(B10_2, find_tol_gamma(Model$Tl[,,i], Model$Te[,,i], Model$Tr[,,i]), n)
}
```

Calculate means, 95% credible intervals, and probabilities of the posterior mean being positive and probabilities from the highest frequency smaller than those from the lowest frequency.

```
mean <-  rowMeans(Beta[,,,], dims = 3)
lower <- upper <- array(NA, c(nrow(Y_star),E,R))
for (freq in 1:nrow(Y_star)){
  for (e in 1:E){
    for (r in 1:R){
      lower[freq,e,r] <- quantile(Beta[freq,e,r,], probs = .025)
      upper[freq,e,r] <- quantile(Beta[freq,e,r,], probs = .975)
    }
  }
```

```
}

prob <- confounding <- matrix(NA, nrow = E, ncol = R)
for (e in 1:E){
  for (r in 1:R){
    prob[e,r] <- mean(Beta[1,e,r,]>0)
    confounding[e,r] <- mean(Beta[n,e,r,]>Beta[1,e,r,])
  }
}
rownames(prob) <- exposure_names
colnames(prob) <- outcome_names
```

`prob` makes a table for Percentages of posterior estimates being greater than zero (like Table 1 in the paper).

```
prob
```

```
##         Hypertension     CKD Hyperlipidemia     CHF Diabetes
## Theme 1      0.72750 0.79125        0.82625 0.86875  0.76750
## Theme 2      0.71375 0.88875        0.88750 0.85125  0.88750
## Theme 3      0.71000 0.80750        0.78125 0.75125  0.75375
## Theme 4      0.77000 0.91250        0.89875 0.85625  0.83000
```

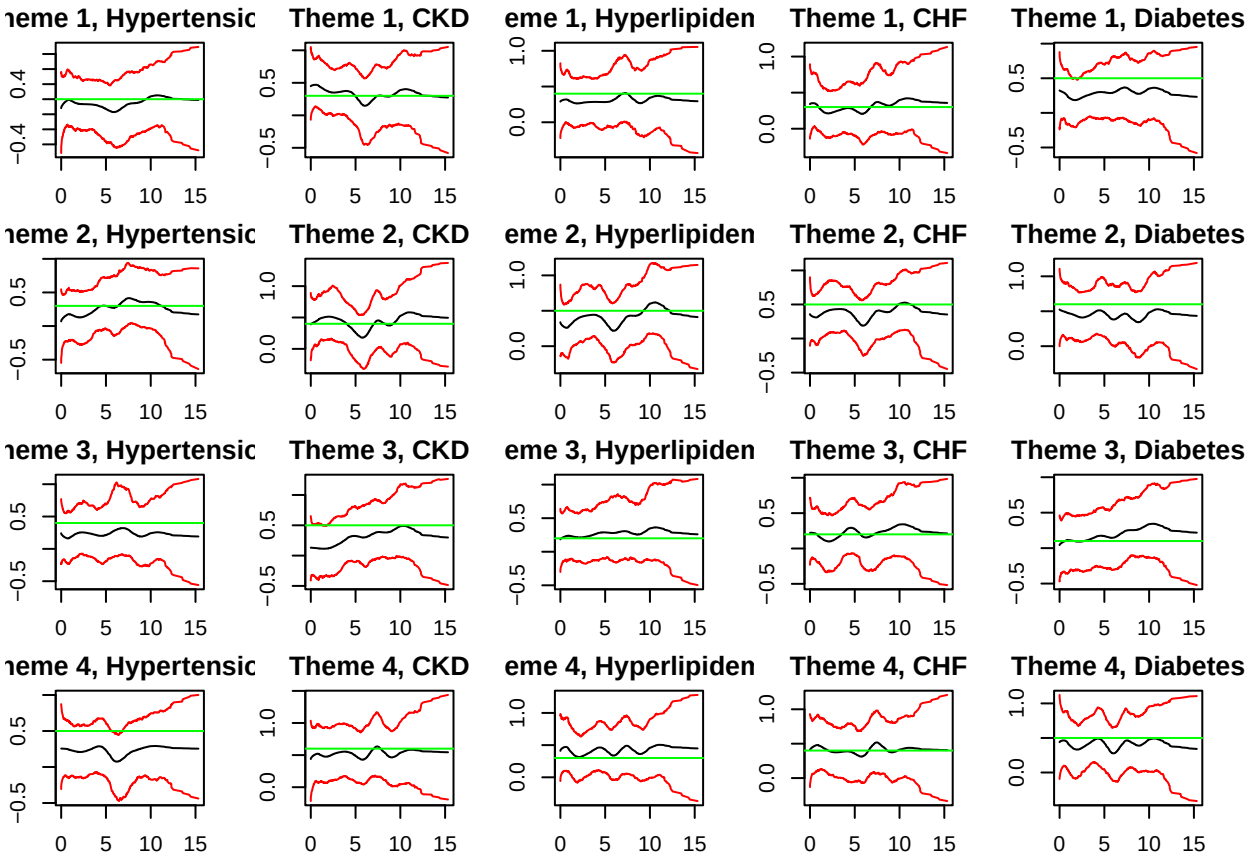
`confounding` makes a table for Probabilities of coefficient estimates at the lowest frequency greater than those at the highest frequency (like Table 2 in the paper).

```
confounding
```

```
##         [,1]    [,2]    [,3]    [,4]    [,5]
## [1,] 0.40625 0.63500 0.49875 0.46875 0.58500
## [2,] 0.37500 0.42625 0.43250 0.51375 0.57875
## [3,] 0.54500 0.36250 0.42000 0.53000 0.33000
## [4,] 0.50250 0.41375 0.47250 0.48500 0.57250
```

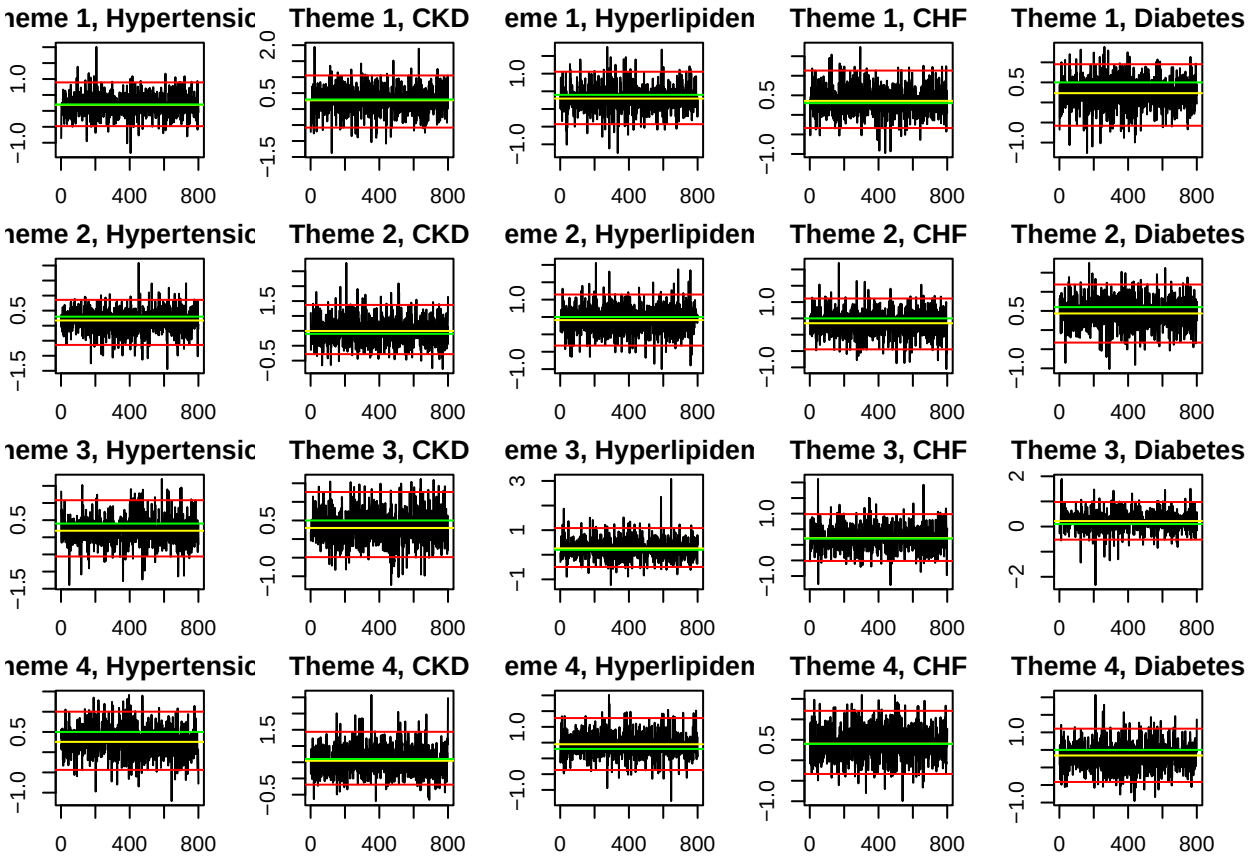
Check result for all frequencies.

```
par(mfrow = c(E,R), mar = c(2,2,2,2))
for (e in 1:E){
  for (r in 1:R){
    plot(W_nc, mean[,e,r], type = "l", main = paste0(exposure_names[e],", ", outcome_names[r]), ylim = c
    lines(W_nc, upper[,e,r], col = "red")
    lines(W_nc, lower[,e,r], col = "red")
    abline(h=sim_beta[e,r], col = "green")
  }
}
```



Check the trace plots.

```
par(mfrow = c(E,R), mar = c(2,2,2,2))
for (e in 1:E){
  for (r in 1:R){
    plot(Beta[1,e,r,], type = "l", main = paste0(exposure_names[e],", ", outcome_names[r]))
    abline(h = mean[1,e,r], col = "yellow")
    abline(h = upper[1,e,r], col = "red")
    abline(h = lower[1,e,r], col = "red")
    abline(h = sim_beta[e,r], col = "green")
  }
}
```



Naive model

```
model_naive <- fit_svi_naive
mean_naive <-  upper_naive <- lower_naive <- matrix(NA, nrow = E, ncol = R)
for (e in 1:E){
  for (r in 1:R){
    mean_naive[e,r] <- mean(model_naive$beta[e,r,])
    lower_naive[e,r] <- quantile(model_naive$beta[e,r,], probs = .025)
    upper_naive[e,r] <- quantile(model_naive$beta[e,r,], probs = .975)
  }
}

rownames(mean_naive) <- rownames(lower_naive) <- rownames(upper_naive) <- exposure_names
colnames(mean_naive) <- colnames(lower_naive) <- colnames(upper_naive) <- outcome_names
```

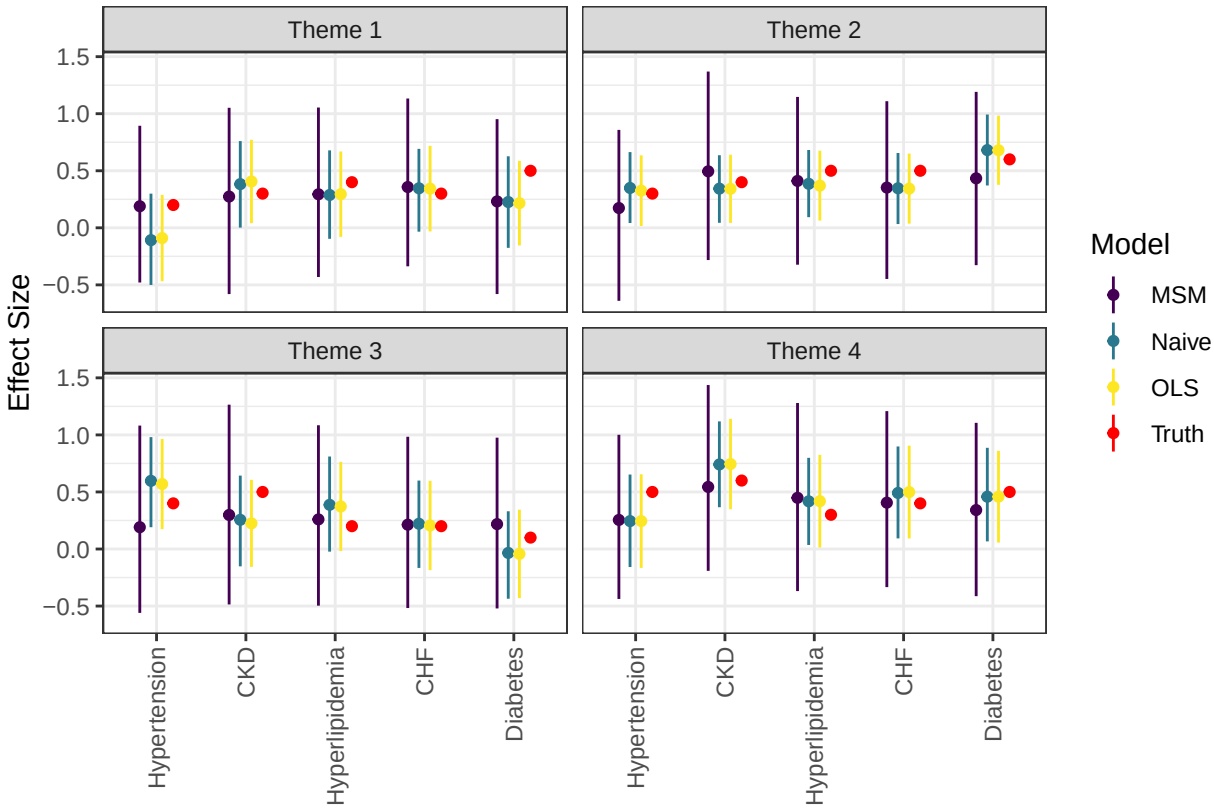
Make plots

Forest plot

```
n <- nrow(X_star)
model_type = c("MSM", "Naive", "OLS", "Truth")
```

```r
model_level <- c(paste0(exposure_names[1], ", ", model_type),
                 paste0(exposure_names[2], ", ", model_type),
                 paste0(exposure_names[3], ", ", model_type),
                 paste0(exposure_names[4], ", ", model_type))

sum <- list()
for (r in 1:R){
  sum_msm_high <- sum_naive <- sum_OLS <- sum_OLS_state <- truth <- list()
  for (e in 1:E){
    sum_msm_high[[e]] <- data.frame(
      Exposure = exposure_names[e],
      Outcome = outcome_names[r],
      Model = paste0("MSM"),
      EffectSize = mean[1,e,r],
      CI_lower = lower[1,e,r],
      CI_upper = upper[1,e,r])

    sum_naive[[e]] <- data.frame(
      Exposure = exposure_names[e],
      Outcome = outcome_names[r],
      Model = paste0("Naive"),
      EffectSize = mean_naive[e,r],
      CI_lower = lower_naive[e,r],
      CI_upper = upper_naive[e,r])

    sum_OLS[[e]] <- data.frame(
      Exposure = exposure_names[e],
      Outcome = outcome_names[r],
      Model = paste0("OLS"),
      EffectSize = summary(fit_svi[[r]])$coefficients[e,1],
      CI_lower = summary(fit_svi[[r]])$coefficients[e,1]-2*summary(fit_svi[[r]])$coefficients[e,2],
      CI_upper = summary(fit_svi[[r]])$coefficients[e,1]+2*summary(fit_svi[[r]])$coefficients[e,2])

    truth[[e]] <- data.frame(
      Exposure = exposure_names[e],
      Outcome = outcome_names[r],
      Model = paste0("Truth"),
      EffectSize = sim_beta[e,r],
      CI_lower = sim_beta[e,r],
      CI_upper = sim_beta[e,r])
  }

  sum[[r]] <- rbind(
    sum_msm_high[[1]],
    sum_naive[[1]],
    sum_OLS[[1]],
    sum_msm_high[[2]],
    sum_naive[[2]],
    sum_OLS[[2]],
    sum_msm_high[[3]],
    sum_naive[[3]],
    sum_OLS[[3]],
    sum_msm_high[[4]],
```

```
    sum_naive[[4]],
    sum_OLS[[4]],
    truth[[1]],
    truth[[2]],
    truth[[3]],
    truth[[4]])

  sum[[r]] <- sum[[r]] %>% arrange(factor(Model, levels = model_level))
}

full_sum <- rbind(sum[[1]],sum[[2]],sum[[3]],sum[[4]],sum[[5]])

ggplot(data = full_sum, aes(x = factor(Outcome, level=outcome_names), y = EffectSize, ymin = CI_lower, y
  geom_pointrange(position=position_dodge(width=.5), size = .2, aes(color = factor(Model, levels = model
  scale_color_manual(name='Model', labels=c('MSM', 'Naive', 'OLS', 'Truth'), values = c("#440154FF", "#
  labs(color = "Model", x = "", y = "Effect Size") +
  # geom_hline(yintercept = 0, linetype = 3) +
  theme_bw()+
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
  facet_wrap(vars(factor(Exposure,level=exposure_names)), nrow = 2)
```

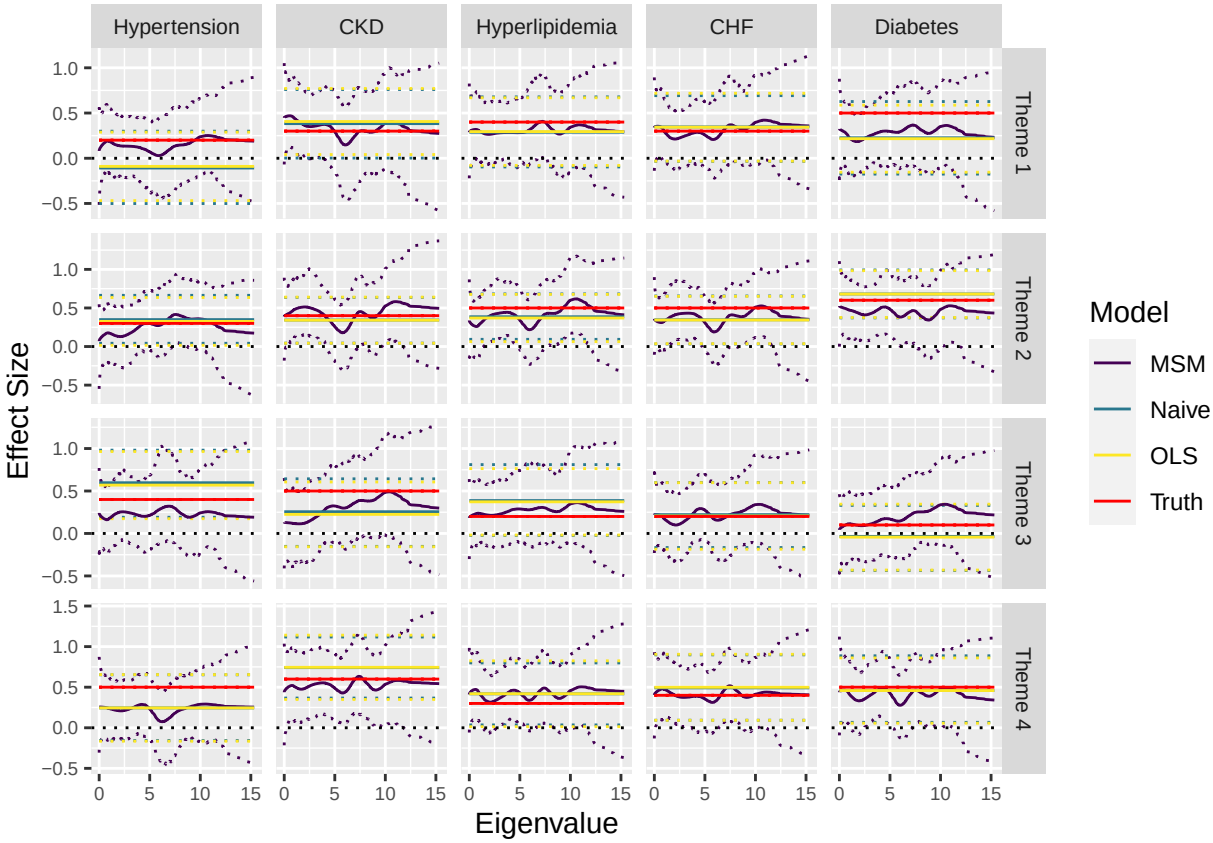


```
# scale_colour_viridis_d()
```

Coefficient by frequency plot

```
curr_data <- data.frame(W = W_nc, Mean = mean[,1,1], CI_upper = upper[,1,1], CI_lower = lower[,1,1], Exp

for (r in 2:5){
  curr_data2 <- data.frame(W = W_nc, Mean = mean[,1,r], CI_upper = upper[,1,r], CI_lower = lower[,1,r],
  curr_data <- rbind(curr_data2, curr_data)
}

for (e in 2:E){
  for (r in 1:5){
    curr_data3 <- data.frame(W = W_nc, Mean = mean[,e,r], CI_upper = upper[,e,r], CI_lower = lower[,e,r]
    curr_data <- rbind(curr_data3, curr_data)
  }
}

for (e in 1:E){
  for (r in 1:R){
    curr_data_naive <- data.frame(
      W = W_nc,
      Mean = mean_naive[e,r],
      CI_upper = upper_naive[e,r],
      CI_lower = lower_naive[e,r],
      Exposure = exposure_names[e],
      Outcome = outcome_names[r],
      Model = paste0("Naive")
    )
    curr_data <- rbind(curr_data_naive, curr_data)
  }
}

for (e in 1:E){
  for (r in 1:R){
    curr_data_ols <- data.frame(
      W = W_nc,
      Mean = summary(fit_svi[[r]])$coefficients[e,1],
      CI_upper = summary(fit_svi[[r]])$coefficients[e,1]+2*summary(fit_svi[[r]])$coefficients[e,2],
      CI_lower = summary(fit_svi[[r]])$coefficients[e,1]-2*summary(fit_svi[[r]])$coefficients[e,2],
      Exposure = exposure_names[e],
      Outcome = outcome_names[r],
      Model = paste0("OLS")
    )
    curr_data <- rbind(curr_data_ols, curr_data)
  }
}

for (e in 1:E){
  for (r in 1:R){
    curr_data_true <- data.frame(
```

```
      W = W_nc,
      Mean = sim_beta[e,r],
      CI_upper = sim_beta[e,r],
      CI_lower = sim_beta[e,r],
      Exposure = exposure_names[e],
      Outcome = outcome_names[r],
      Model = paste0("True")
    )
    curr_data <- rbind(curr_data_true, curr_data)
  }
}



curr_data_long <- curr_data %>% pivot_longer(cols = c("Mean","CI_upper","CI_lower"), names_to = "line_ty

ggplot(data = curr_data_long, aes(x=W, y=estimate, linetype = line_type, color = Model)) +
  geom_line() +
  scale_color_manual(name='Model', labels=c('MSM', 'Naive', 'OLS', 'Truth'), values = c("#440154FF", "#
  labs(x="Eigenvalue",y="Effect Size") +
  scale_linetype_manual(values = c("Mean" = "solid", "CI_upper" = "dotted", "CI_lower" = "dotted")) +
  geom_hline(yintercept = 0, linetype = 3) +
  facet_grid(cols=vars(factor(Outcome,level=outcome_names)),rows=vars(factor(Exposure,level=exposure_nam
  theme(strip.text = element_text(size=8),
        axis.text = element_text(size=7),
        legend.position = "right") +
  # geom_hline(yintercept = , linetype = 3) +
  guides(linetype = "none")
```


Hypertension
CKD
Hyperlipidemia
CHF
Diabetes
Theme 1
Theme 2
Theme 3
Theme 4
Effect Size
Eigenvalue
Model
MSM
Naive
OLS
Truth